

Theorem 4.1

任何维护 resizable array 的数据结构，在某些时刻都必须使用

$$N + \Omega(\sqrt{N})$$

空间。即使只考虑 grow 和 access 操作，这个下界也成立。

该引理给出的范围很强“任何维护...”

直觉：Brodnik 的数据结构给出了 $N + O(\sqrt{N})$ 大小的空间，

平方与开方中暗含的 $\frac{1}{2}$ 可能是某种最优结构

证明：我们将总情况分为两类：

N 次 grow 后， N 个元素， $\rightarrow k$ 个内存块

case 1:

$$k \geq \sqrt{N}$$

显然，存储这些 k 个块的指针需要大小 k 。

原数据大小为 N ，故总空间至少为 $k + N \geq N + \sqrt{N}$

case 2:

$$k < \sqrt{N}$$

此时根据抽屉原理必有一个块大小大于 $\sqrt{N}$ 。

设该块刚申请出来时总元素个数为 N' ，

则当时总空间至少为 $N' + \sqrt{N} \geq N' + \sqrt{N'}$

综上, 存在某一时刻, 总空间为 $N + \Omega(\sqrt{N})$

从上述证明可看出何为平时空间, 何为临时空间,

case 1 证明所用的为平时空间, case 2 所用的是临时空间
关键在于考虑“某一刻突然申请了一块很大的空间! 而这
刚申请的空间就是临时空间, 至此, 我们可以理解作者为何要区
分这两种空间!

Theorem 4.2

任何 $(s(N), t(N))$ -implementation 都必须满足:

$$s(N)t(N) \geq N$$

即使只支持 grow 和 access 操作也成立。

证明: 思路与 Th 4.1 保持一致. 考虑抽屉原理, 至少有一个 block 的大小不小于 $\frac{N}{s(N)}$ (注: $s(N)$ 代表指针/块个数), 刚分配出来这部分空间时, 临时额外-空间是一个 block 大小, 故 $t(N) \geq \frac{N}{s(N)}$

$$\Rightarrow s(N)t(N) \geq N$$

注: 很明显能看出 Th 4.2 是 Th 4.1 的加强, 因为在 Th 4.2 中取 $s(N) = t(N) = \sqrt{N}$ 即为 Th 4.1

如果我们设 $s(N) = O(N^{\frac{1}{r}})$
那么 $t(N) \geq \Omega(N^{1-\frac{1}{r}})$

以上给出的两个定理给出了 $s(N)t(N)$ 的理论下界,
这意味着我们有希望围绕这个下界给出构造

所以, 核心方法部分作者的基本思路就是先通过这两个引理找到
理论最优, 然后给出 r 从特殊到一般的构造.

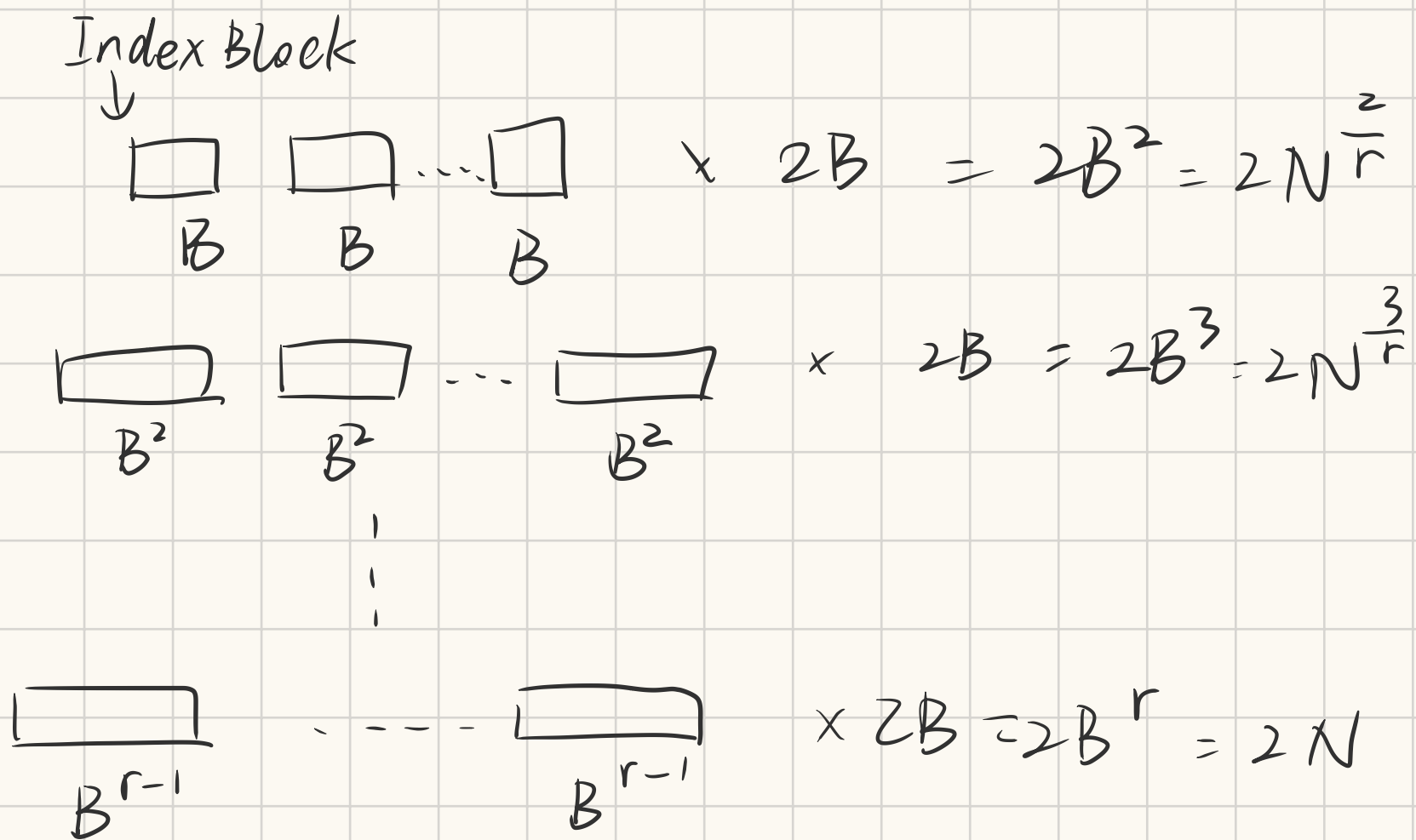
构造方面, 最基本的 $r=2$ 的构造已经由 Brodnik 等人
给出, 在后面, 作者依次又给出了 $r=3$, 以及一般 r
的构造

本文一般方法上界分析

一、对于一般 r 的构造, 证明: (Th 6.1)

(1) grow/shrink 均摊成本为 $O(r)$

注: 一般的构造指的是 $(O(N^{\frac{1}{T}}), O(N^{\frac{1}{T}}))$ - impl
 $\downarrow$ $\downarrow$
 存储 额外空间 存储 临时空间



4) 前面方法构造其实已经隐含了这个证明思想，
因为最多有 r 层，所以对于任意一个元素，它最坏需要
下沉 $O(r)$ 次（比如第 $1, 2, \dots, r-2$ 层全满了），所以
总代价至多为 $O(Nr)$ ，但因为共有 N 个元素，均摊下

来 每个元素代价就至少为 $O(r)$

以上是一种直观理解。如果严谨证明的话，可以使用以下的方法：

使用之前提到过的记账法，为 level i 中的元素分配 $r-i$ 个 credit。当某个元素下沉复制一次时，代价为 1，credit 变为 $r-(i+1)$ ，初始插入元素时，分配的 credit 为 $(r-1)$ ，最终 credit ≥ 0 ，所以 Grow 的均摊成本为 $O(r)$

→ shrink.

(2) 先回忆一下 shrink 的操作。

由于 shrink 始终去删除末尾元素（最新插入的元素），所以应该自顶向下找，如果没有 $size-B$

block, 就要 split. 我们区分两种 case

1. 最后一个 B-size block 还有元素

这样直接删除即可, 代价为 0

2. 进行一次 B^k 代价的复制, 然后删除.

$$\uparrow$$
$$(B^k \approx (B + B^2 + \dots + B^{k-1})(B-1))$$

我们考虑两次 split 中间, 因为一下拆出 B^k 个元素, 所以同级的两个 split 中间必有 B^k 个 shrink.

假设一次 shrink 获得 3 个 credit.

则这 level k 至少积了 $3B^k$ 个

成本包括两部分, 1. 复制成本 2. 给移动后
的元素补充 credit

↓

$$B(k-1) + (B-1) \sum_{i=1}^{k-1} (k-i) B^i$$

$$\leq B \sum_{i=1}^{k-1} (k-i) B^i$$

$$= B \sum_{j=1}^{k-1} j B^{k-j}$$

$$\leq B^{k+1} \sum_{j \geq 1} j B^{-j}$$

$$\text{无穷级数} \approx \frac{B}{(B-1)^2}$$

$$\leq \frac{B^{k+2}}{(B-1)^2} \leq 2B^k$$

所以飞姐可以支付！

Rebuild:

同样可以由积分 credit 支付。

